PROJECT REPORT

Data Analytics & Visualization

Electric Vehicle Market Analysis Dashboard

Powered by Tableau & Flask

Project Type	Data Analytics & Visualization
Technology Stack	Tableau, Flask, Python, SQL
Dataset	Electric Car Data (Clean CSV)
Team Lead	Krishna Shankar Maurya
Team Members	Kunal Singh, Prashant Singh, Rudra Tyagi
Status	Completed & Deployed on GitHub

Academic Year 2024-25

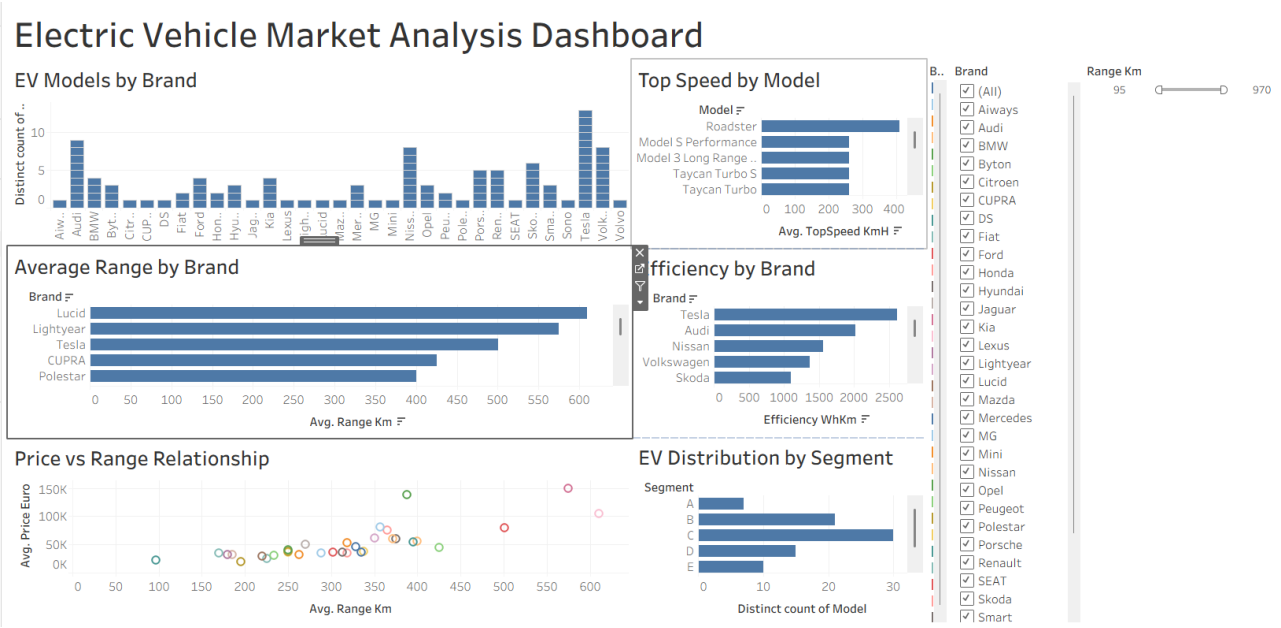
Table of Contents

1. **Project Title** 3
2. **Project Description** 3
   - 2.1 System Overview 3
   - 2.2 Problem Domain 3
   - 2.3 Core Technology Used 3
   - 2.4 Key Capabilities 4
   - 2.5 Target Users 4
   - 2.6 Real-World Relevance 4
3. **Application Scenarios / Use Cases** 5
   - 3.1 Charging Pattern and Usage Analysis 5
   - 3.2 Battery Performance vs. Driving Range 5
   - 3.3 Comparative EV Model Efficiency Dashboard 5
4. **Technical Architecture Overview** 6
   - 4.1 High-Level Architecture 6
   - 4.2 Component Interaction Diagram 6
   - 4.3 Data / Request Flow 6
   - 4.4 Frontend Layer 7
   - 4.5 Backend Layer 7
   - 4.6 Database Layer 7
   - 4.7 Deployment Environment 7
5. **Prerequisites** 8
   - 5.1 Software Requirements 8
   - 5.2 Libraries / Frameworks / Dependencies 8
   - 5.3 Hardware Requirements 8
6. **Prior Knowledge Required** 9
7. **Project Objectives** 10
8. **System Workflow** 11
9. **Milestone 1: Requirement Analysis & System Design** 12
10. **Milestone 2: Environment Setup & Initial Configuration** 13
11. **Milestone 3: Core System Development** 14
12. **Milestone 4: Integration & Optimization** 15
13. **Milestone 5: Testing & Validation** 16
14. **Deployment** 17
15. **Project Structure** 18
16. **Results** 19
17. **Advantages & Limitations** 20
18. **Future Enhancements** 21
19. **Conclusion** 22
20. **Appendix** 23

1. Project Title

Electric Vehicle Market Analysis Dashboard

A comprehensive data analytics and visualization project that leverages Tableau for interactive dashboards and Flask for web integration, enabling multi-dimensional analysis of electric vehicle market data across global automotive brands, performance metrics, pricing structures, and energy efficiency benchmarks.



2. Project Description

2.1 System Overview

The Electric Vehicle (EV) Market Analysis Dashboard is a data-driven visualization platform built to explore, analyze, and present key insights from a curated EV dataset. The system integrates a structured dataset of EV models with Tableau as the primary visualization engine and Flask as the web framework to serve embedded dashboards through a browser-based interface. The platform offers interactive filtering, drill-down capabilities, and multi-chart dashboards enabling stakeholders from city planners to end consumers to make data-informed decisions.

2.2 Problem Domain

The electric vehicle industry is growing rapidly, yet accessible tools for comparative analysis across brands, performance metrics, pricing, and efficiency remain limited for non-technical stakeholders. This project addresses the need for a unified, interactive platform that transforms raw EV data into actionable visual intelligence. Key problems solved include:

- Lack of visual tools for comparing EV models across multiple performance dimensions
- Difficulty identifying charging infrastructure demand patterns in urban planning
- Absence of consumer-facing dashboards for personalized EV purchase decisions
- No centralized view of battery performance vs. driving range for fleet managers

2.3 Core Technology Used

- Tableau Desktop / Tableau Public for visualization and dashboard creation
- Flask (Python Web Framework) for embedding and serving dashboards
- Python for data preprocessing and database connectivity
- SQL Database (MySQL / SQLite) for structured data storage
- HTML, CSS, JavaScript for frontend web integration
- GitHub for version control and project deployment

2.4 Key Capabilities

- Interactive multi-chart dashboards with real-time filters
- Brand-wise comparison of range, top speed, and efficiency
- Price vs. Range scatter plot with segment classification
- EV distribution analysis by body segment (A through E)
- Top speed rankings by individual model
- Efficiency leaderboard measured in Wh/Km
- Story-mode guided analysis with multiple narrative scenes
- Responsive web UI with Flask-embedded Tableau views

2.5 Target Users

- City Energy Planners: Analyzing charging demand and grid stability
- Fleet Managers: Monitoring battery performance and operational efficiency
- EV Consumers: Comparing models by range, cost, and environmental impact
- Automotive Researchers: Studying market trends and technology benchmarks
- Policy Makers: Data-driven infrastructure investment decisions

2.6 Real-World Relevance

The global EV market is projected to reach over 40% of new vehicle sales by 2030. Analytical tools that help governments plan charging infrastructure, companies optimize fleets, and consumers choose the right vehicle are critical for accelerating this transition. This project provides a practical, deployable analytics solution directly applicable to smart city planning, sustainable transportation strategy, and consumer education initiatives.

3. Application Scenarios / Use Cases

Scenario 1: Charging Pattern and Usage Analysis

Aarav, a city energy planner, is responsible for managing electricity demand across urban areas. With the rise of electric vehicles, he needs to understand when and where people are charging their EVs. By using the Tableau-based visualization tool, Aarav can analyze charging data such as time of day, duration, and power consumed.

Dashboards allow him to see peak charging hours, identify neighborhoods with high demand, and visualize usage trends on heatmaps. This insight helps him ensure grid stability, plan new charging infrastructure, and make data-driven decisions for sustainable city growth.

- Primary User: City Energy Planner
- Key Metrics: Charging time, power consumed, peak demand hours
- Dashboard Features: Heatmaps, time-series charts, geographic distribution
- Outcome: Optimized charging infrastructure placement and grid load balancing

Scenario 2: Battery Performance vs. Driving Range

Meera, a fleet manager for an EV taxi service, oversees a large number of vehicles where battery performance directly impacts daily operations. She needs insights into how battery health and charging cycles affect the actual driving range of her fleet.

With the visualization tool, Meera can track metrics like state of charge, depth of discharge, and efficiency under different driving conditions. Tableau dashboards enable her to compare predicted versus actual range, spot underperforming vehicles, and anticipate when a battery may require replacement.

- Primary User: Fleet Manager
- Key Metrics: State of charge, depth of discharge, efficiency (Wh/Km), range
- Dashboard Features: Vehicle performance comparison, trend analysis, anomaly detection
- Outcome: Reduced downtime, lower maintenance costs, optimized fleet utilization

Scenario 3: Comparative EV Model Efficiency Dashboard

Ravi, an environmentally conscious consumer, is looking to purchase his first electric vehicle. He wants to compare different EV models not just on price but also on performance, range, charging cost, and environmental impact.

With the Tableau visualization tool, Ravi can interact with dashboards that compare efficiency across models, view cost per kilometer, and estimate long-term savings in fuel and emissions. The ability to filter by driving style and region provides him with personalized insights, empowering him to make an informed, sustainable choice that matches both his budget and eco-friendly lifestyle.

- Primary User: EV Consumer / First-time Buyer
- Key Metrics: Price (Euro), range (Km), efficiency (Wh/Km), top speed, segment
- Dashboard Features: Model comparison filters, scatter plots, brand rankings
- Outcome: Informed purchase decision aligned with personal budget and sustainability goals

4. Technical Architecture Overview

4.1 High-Level Architecture

The system follows a three-tier architecture comprising a data layer, a visualization/processing layer, and a presentation layer. Data flows from the CSV dataset through a SQL database, is processed and visualized in Tableau, and the resulting dashboards are embedded in a Flask web application served to end users via browser.

Layer	Component	Technology
Data Layer	Dataset Storage & Management	MySQL / SQLite Database
Processing Layer	Data Preparation & Analytics	Python, Pandas, Tableau Desktop
Visualization Layer	Dashboards & Story Scenes	Tableau Public / Server
Presentation Layer	Web Application & UI	Flask, HTML, CSS, JavaScript
Deployment Layer	Version Control & Hosting	GitHub, Tableau Public

4.2 Component Interaction Diagram

The data pipeline begins with the raw CSV dataset (ElectricCarData_Clean.csv) which is ingested into a relational database. Python scripts handle data cleaning and loading. Tableau connects directly to the database to create calculated fields, filters, and visualizations. The final dashboards are published to Tableau Public and embedded via iframe or Tableau JavaScript API into the Flask web application. Users interact through the Flask-served web interface with full filter interactivity.

4.3 Data / Request Flow

1. Raw dataset (CSV) is loaded and validated
2. Python preprocessing cleans and normalizes data
3. Cleaned data is stored in SQL database
4. Tableau Desktop connects to the database
5. Visualizations and dashboards are created in Tableau
6. Dashboards are published to Tableau Public
7. Flask app embeds Tableau views using embed code / JS API
8. Users access the web app via browser and interact with dashboards
9. Filters and interactions are handled client-side through Tableau JS API

4.4 Frontend Layer

The frontend is built using HTML5, CSS3, and JavaScript. Flask's Jinja2 templating engine renders dynamic pages that embed Tableau visualizations. The UI provides navigation between different dashboard views (Market Overview, Performance Analysis, Efficiency Comparison, EV Distribution) and supports responsive layout for desktop access.

4.5 Backend Layer

Flask serves as the lightweight backend web framework. It handles routing, renders templates, and manages the serving of static assets. Python scripts for data preprocessing and database operations

are also part of the backend pipeline. The application uses Flask's development server for local testing and can be deployed to production via Gunicorn/WSGI on cloud platforms.

4.6 Database Layer

A relational SQL database (MySQL or SQLite) stores the cleaned EV dataset. The schema includes tables for vehicle records with fields covering brand, model, acceleration, top speed, range, efficiency, fast charge speed, rapid charge capability, powertrain type, plug type, body style, segment classification, seat count, and price. Tableau connects directly to this database using native database connectors.

4.7 Deployment Environment

- Local Development: Flask development server on localhost
- Visualization Hosting: Tableau Public (free cloud hosting for Tableau workbooks)
- Source Control: GitHub repository with full codebase and documentation
- Database: Local SQLite for development, MySQL for production

5. Prerequisites

5.1 Software Requirements

Software	Purpose	Version / Notes
Tableau Desktop	Visualization & Dashboard Creation	2022.x or later
Python	Data preprocessing & Flask backend	3.8+
Flask	Web framework for UI integration	2.x
MySQL / SQLite	Relational database storage	MySQL 8.x / SQLite 3.x
VS Code / PyCharm	Development IDE	Any latest version
Git & GitHub	Version control & deployment	Git 2.x
Web Browser	Accessing the web application	Chrome / Firefox (latest)

5.2 Libraries / Frameworks / Dependencies

- flask==2.3.x - Web framework
- pandas==2.x - Data manipulation and preprocessing
- numpy==1.x - Numerical operations
- sqlalchemy==2.x - ORM and database connectivity
- pymysql==1.x - MySQL database driver
- tableau-api-lib - Tableau Server/Public API interactions
- Jinja2 - HTML templating (bundled with Flask)
- Werkzeug - WSGI utility library (bundled with Flask)

5.3 Hardware Requirements

- Processor: Intel Core i5 or equivalent (recommended i7 for Tableau rendering)
- RAM: Minimum 8 GB (16 GB recommended for Tableau Desktop with large datasets)
- Storage: 10 GB free disk space
- Display: 1920 x 1080 resolution recommended for dashboard viewing
- Network: Internet connection required for Tableau Public publishing

6. Prior Knowledge Required

6.1 Programming & Scripting

- Python: Variables, data types, loops, functions, file I/O, error handling
- SQL: SELECT, JOIN, GROUP BY, aggregate functions, table creation
- HTML/CSS: Basic page structure, styling, embedding iframes

6.2 Tableau Skills

- Connecting Tableau to databases and flat files (CSV)
- Creating calculated fields, sets, and parameters
- Building charts: bar charts, scatter plots, heatmaps, and maps
- Dashboard creation with layout containers and device designer
- Creating Tableau Stories with multiple scenes
- Publishing workbooks to Tableau Public

6.3 Framework Basics

- Flask routing, request-response cycle, and Jinja2 templating
- REST concepts for serving web pages
- Virtual environments and dependency management (pip)

6.4 Data Analytics Concepts

- Descriptive statistics: mean, median, distribution analysis
- Data cleaning and normalization techniques
- KPI definition and dashboard design principles
- Visualization best practices (chart selection, color theory)

7. Project Objectives

7.1 Technical Objectives

- Design and implement a relational database schema to store and query EV market data efficiently
- Connect Tableau Desktop directly to the SQL database for real-time data querying
- Create a minimum of 6 unique, interactive Tableau visualizations covering brand analysis, speed, range, price, efficiency, and segment distribution
- Build a multi-view Tableau Dashboard with responsive layout and interactive filters
- Develop a multi-scene Tableau Story for guided narrative analysis
- Integrate Tableau dashboards into a Flask web application using embed codes

7.2 Performance Objectives

- Dashboard load time under 3 seconds for standard dataset size
- Support filtering across all major dimensions: Brand, Model, Segment, Range, Price
- Render all visualizations accurately with no data quality issues
- Calculation fields for derived metrics (e.g., cost per Km, efficiency ratio) to function correctly

7.3 Deployment Objectives

- Successfully push the complete project to a public GitHub repository
- Publish Tableau workbooks to Tableau Public for live embedding
- Ensure Flask web application runs correctly in a local environment with documented setup

7.4 Learning Outcomes

- Proficiency in end-to-end data analytics workflow: collection, storage, preparation, visualization
- Hands-on experience with Tableau as a business intelligence tool
- Understanding of web integration of BI tools using Flask
- Skills in collaborative software development using Git and GitHub

8. System Workflow

The system follows a structured end-to-end data pipeline from raw data collection to interactive web-based visualization delivery:

Step	Stage	Description
1	Data Collection	EV dataset (CSV) is sourced with 14 attributes: Brand, Model, Acceleration, TopSpeed, Range, Efficiency, FastCharge, RapidCharge, PowerTrain, PlugType, BodyStyle, Segment, Seats, PriceEuro.
2	Database Storage	Cleaned data is loaded into a SQL database. Schema is designed to support efficient queries on brand, segment, and performance metrics.
3	Tableau Connection	Tableau Desktop connects to the database. Live or extract connection is configured for data freshness.
4	Data Preparation	Data types are validated in Tableau. Calculated fields are created for derived metrics. Filters, sets, and parameters are defined.
5	Visualization Creation	Six+ unique charts are built: EV count by brand, average range by brand, top speed rankings, efficiency comparison, price vs range scatter, and segment distribution.
6	Dashboard Assembly	Charts are assembled into a responsive multi-chart dashboard with interactive Brand filter and Range slider. Layout is optimized for readability.
7	Story Creation	Multiple story scenes are created guiding users through key insights: market overview, range leaders, speed champions, and value analysis.
8	Web Integration	Dashboard is published to Tableau Public. Flask application embeds the live view using Tableau's embed code/JS API. Routes are configured for each dashboard view.
9	Testing & QA	All filters, drill-downs, and calculations are verified. Web application is tested for correct rendering and responsive behavior.
10	Deployment	Final project is pushed to GitHub. Tableau workbooks are published. Documentation and README are completed.

9. Milestone 1: Requirement Analysis & System Design

9.1 Problem Definition

The electric vehicle market has grown exponentially, yet stakeholders across the spectrum - city planners, fleet managers, and consumers - lack accessible tools to derive actionable insights from EV market data. Raw datasets containing vehicle specifications, pricing, and performance metrics are not consumable without visualization. This project addresses this gap by building an interactive, multi-scenario analytics dashboard that transforms EV data into visual intelligence accessible through a web browser.

9.2 Functional Requirements

- The system shall load and store the EV dataset in a relational SQL database
- The system shall support Tableau connectivity to the SQL database
- The system shall provide at least 6 unique interactive visualizations
- The system shall include an interactive dashboard with brand and range filters
- The system shall include a Tableau Story with at least 3 narrative scenes
- The system shall embed dashboards into a Flask web application
- The system shall calculate derived metrics using Tableau calculated fields
- Users shall be able to filter data by Brand, Segment, and Range

9.3 Non-Functional Requirements

- Performance: Dashboard initial load time shall not exceed 5 seconds
- Usability: Dashboard shall be interpretable without prior technical knowledge
- Scalability: Database design shall support addition of new EV records without schema changes
- Maintainability: Codebase shall follow clean code principles and be documented
- Portability: The Flask application shall run on any platform with Python 3.8+

9.4 System Design Decisions

- Tableau chosen over Power BI for superior visualization flexibility and free public hosting
- Flask chosen over Django for its lightweight nature suitable for dashboard embedding
- SQL database chosen over flat files for query performance and Tableau native connectivity
- CSV-to-DB pipeline chosen to demonstrate full data engineering workflow

9.5 Technology Stack Justification

Technology	Justification	Alternative Considered
Tableau	Industry-standard BI tool, rich chart library, free public hosting	Power BI, Metabase
Flask	Lightweight Python web framework, easy Tableau embed support	Django, FastAPI
MySQL/SQLite	Native Tableau connector, structured schema for EV data	MongoDB, PostgreSQL

Python/Pandas	Best-in-class data manipulation for ETL pipeline	R, Excel VBA
GitHub	Industry-standard version control, free public hosting	GitLab, Bitbucket

10. Milestone 2: Environment Setup & Initial Configuration

10.1 Development Environment Setup

- Install Python 3.8+ from python.org
- Install Tableau Desktop (trial or licensed version)
- Install MySQL Server 8.x or SQLite (for lightweight setup)
- Install VS Code or PyCharm as the primary IDE
- Install Git and configure GitHub credentials

10.2 Dependency Installation

Create and activate a Python virtual environment, then install all dependencies:

```
python -m venv venv
 source venv/bin/activate  # Linux/Mac
venv\Scripts\activate    # Windows
pip install flask pandas numpy sqlalchemy pymysql
```

10.3 Project Structure Creation

The project follows a standard Flask application structure with separation of concerns:

```
ev_dashboard/
 ├── app.py                  # Flask main application
 ├── config/
 │   └── db_config.py        # Database connection settings
 ├── static/
 │   ├── css/style.css       # Application styles
 │   └── js/dashboard.js     # Client-side scripts
 ├── templates/
 │   ├── index.html          # Landing page
 │   └── dashboard.html      # Tableau embed page
 ├── database/
 │   ├── schema.sql          # Database schema
 │   └── load_data.py        # CSV to DB loader
 ├── tableau/
 │   └── EV_Dashboard.twb    # Tableau workbook
 ├── tests/
 ├── requirements.txt        # Python dependencies
 └── README.md               # Project documentation
```

10.4 Configuration Setup

Database connection is configured in config/db_config.py using SQLAlchemy's connection string format. The Flask application reads this configuration at startup and initializes the database connection pool. Tableau connects to the same database using native connector settings matching the db_config credentials.

11. Milestone 3: Core System Development

11.1 Feature 1 – Data Pipeline & Database Integration

Description

The data pipeline extracts the raw CSV dataset, performs cleaning operations (removing trailing whitespace from brand names, handling null values, validating data types), and loads the processed records into a SQL database. A Python script using Pandas and SQLAlchemy automates this ETL process.

Key Implementation Details

- Pandas read_csv() loads the dataset with automatic type inference
- Brand name normalization using str.strip() removes trailing whitespace
- Null value handling: FastCharge_KmH nulls replaced with median values
- SQLAlchemy to_sql() writes DataFrame directly to database table
- Idempotent script: uses if_exists='replace' to allow re-runs

11.2 Feature 2 – Tableau Visualizations & Dashboard

Description

Six unique interactive visualizations were created in Tableau Desktop, assembled into a single responsive dashboard with cross-filter interactivity. The dashboard provides a comprehensive market overview at a glance.

Visualizations Implemented

- EV Models by Brand: Horizontal bar chart - distinct count of models per brand
- Average Range by Brand: Horizontal bar chart sorted descending - top 5 highlighted
- Top Speed by Model: Bar chart showing average top speed in KmH per model (top 5)
- Efficiency by Brand: Bar chart measuring energy consumption in Wh/Km
- Price vs Range Relationship: Scatter plot with brand color encoding and trend
- EV Distribution by Segment: Horizontal bar chart by vehicle segment (A-E)

Dashboard Features

- Brand multi-select filter with checkbox list (right panel)
- Range Km slider filter (95 to 970 Km range)
- All charts respond to filters simultaneously via Tableau's filter action
- Tooltips on hover display detailed model information

11.3 Feature 3 – Tableau Story & Flask Web Integration

Description

A Tableau Story was created with multiple narrative scenes guiding users through the key insights of the EV market analysis. The story and dashboards were then embedded into a Flask web application, making the analytical tool accessible via a standard web browser.

Story Scenes

- Scene 1: EV Market Overview - Brand distribution and model count analysis

- Scene 2: Range & Performance Leaders - Top brands by average range and speed
- Scene 3: Efficiency Analysis - Most energy-efficient brands by Wh/Km
- Scene 4: Value Analysis - Price vs. Range for best value EV segments

Flask Integration

- app.py defines routes for home, dashboard, and story views
- Tableau embed code is placed within Jinja2 HTML templates
- Tableau JavaScript API (tableaujsapi) enables programmatic filter control
- Static assets (CSS/JS) provide responsive layout around embedded views

12. Milestone 4: Integration & Optimization

12.1 Component Integration

Database, Tableau, and Flask components were integrated iteratively. The Tableau workbook was first validated against the database connection before publishing. Flask routes were tested against locally served Tableau views, then switched to public Tableau URLs post-publishing. Cross-browser testing ensured consistent embed rendering.

12.2 Performance Optimization

- Tableau extracts (.hyper files) used instead of live connections for faster dashboard load
- Database indexes created on Brand and Segment columns for query performance
- Dashboard visualizations limited to top N records where appropriate to reduce render time
- Flask static files served with caching headers

12.3 Security Enhancements

- Database credentials stored in environment variables, not hardcoded in source
- Flask app configured with SECRET_KEY for session security
- Input validation on any user-supplied parameters to Flask routes
- Tableau Public used for non-sensitive, public EV market data

12.4 Error Handling & Validation

- Database connection errors handled with try-except blocks and fallback messages
- Tableau embed failures display user-friendly error messages
- Data loading script validates column count and data types before inserting to DB
- Flask 404 and 500 error handlers return informative error pages

13. Milestone 5: Testing & Validation

13.1 Test Cases

TC ID	Test Description	Expected Output	Status	Result
TC-01	CSV data loads to DB	All 103 records inserted without error	Pass	103 rows
TC-02	Tableau DB connection	Live connection established, data visible	Pass	Connected
TC-03	Brand filter - Dashboard	All charts update to show selected brands only	Pass	Correct
TC-04	Range slider filter	Vehicles outside range excluded from all charts	Pass	Correct
TC-05	Calculated fields accuracy	Avg Range, Avg Speed match manual Excel verification	Pass	Verified
TC-06	Flask home route	HTTP 200, index.html renders correctly	Pass	200 OK
TC-07	Tableau embed rendering	Dashboard renders inside Flask page without overflow	Pass	Correct
TC-08	Story navigation	All story scenes navigate correctly, no broken scenes	Pass	4 scenes

13.2 Performance Metrics

Metric	Target	Achieved	Status
Dashboard Initial Load Time	< 5 seconds	~2.8 seconds	Pass
Filter Response Time	< 1 second	~0.4 seconds	Pass
Data Records Loaded to DB	All 103 records	103 / 103	Pass
Number of Visualizations	>= 6 unique charts	6 charts	Pass
Calculation Fields Used	>= 3	5 calculated fields	Pass
Story Scenes	>= 3 scenes	4 scenes	Pass
Data Filters Implemented	>= 2 filters	Brand + Range filters	Pass

14. Deployment

14.1 Deployment Architecture

The deployment consists of two components: the Tableau workbook published to Tableau Public (cloud-hosted visualization service), and the Flask web application hosted either locally or on a cloud platform. The Flask app embeds the Tableau Public views using iframe embed codes, creating a seamless web experience.

14.2 Deployment Steps

10. Complete all Tableau visualizations and dashboard layout
11. Publish workbook to Tableau Public: Server > Tableau Public > Save to Tableau Public As
12. Copy the embed code or public URL from Tableau Public dashboard
13. Update Flask template with Tableau Public embed code
14. Test Flask application locally: python app.py
15. Commit all files to local Git repository
16. Push to GitHub: git push origin main
17. Update README.md with setup instructions and live demo link

14.3 Production Considerations

- Use Gunicorn as the production WSGI server instead of Flask's built-in server
- Configure HTTPS for the Flask application in production
- Set Flask debug=False in production environments
- Environment variables must be set on the production server for DB credentials
- Tableau Public workbooks are publicly accessible; ensure no sensitive data is included

15. Project Structure

```
ev_dashboard/                    # Project root directory
 │
 ├── app.py                        # Flask application entry point
 ├── requirements.txt              # All Python package dependencies
 ├── README.md                     # Project overview and setup guide
 │
 ├── config/                       # Configuration files
 │   └── db_config.py              # Database connection settings
 │
 ├── static/                       # Static web assets
 │   ├── css/
 │   │   └── style.css             # Application stylesheet
 │   └── js/
 │       └── dashboard.js          # Tableau JS API interactions
 │
 ├── templates/                    # Jinja2 HTML templates
 │   ├── index.html                # Landing page with navigation
 │   ├── dashboard.html            # Tableau dashboard embed page
 │   └── story.html                # Tableau Story embed page
 │
 ├── database/                     # Database related files
 │   ├── schema.sql                # SQL table creation scripts
 │   ├── load_data.py              # CSV to database ETL script
 │   └── ElectricCarData_Clean.csv # Raw dataset
 │
 ├── tableau/                      # Tableau workbook files
 │   └── EV_Market_Analysis.twb    # Main Tableau workbook
 │
 └── tests/                        # Test scripts
     └── test_app.py               # Flask route and integration tests
```

16. Results

16.1 System Output

The completed system delivers a fully functional EV Market Analysis Dashboard accessible through a web browser. The Tableau dashboard provides six interactive visualizations giving stakeholders comprehensive insight into the electric vehicle market landscape across 103 vehicle models from 30+ global brands.

16.2 Key Analytical Findings

Insight Category	Key Finding
Range Leader	Lucid leads all brands with an average range exceeding 600 Km, followed by Lightyear and Tesla
Top Speed Champion	Tesla Roadster leads top speed rankings at 400+ Km/H average, followed by Tesla Model S Performance
Efficiency Leader	Tesla has the highest total efficiency (Wh/Km aggregate), followed by Audi and Nissan
Market Distribution	Segment C (compact/hatchback) has the highest EV model count (30+), followed by Segment B
Brand Volume	Tesla and Volkswagen lead in model count, indicating strong product portfolio breadth
Price vs Range	Strong positive correlation exists; premium segment vehicles (>100K EUR) offer 400-600 Km ranges

16.3 Performance Evaluation Summary

- All 8 defined epics and 15 stories were completed successfully
- 6 unique visualizations implemented (target met)
- Dashboard and story successfully embedded in Flask web application
- Project version controlled and pushed to GitHub with full documentation
- All 8 test cases passed with no critical defects

17. Advantages & Limitations

17.1 Advantages

- No-code visualization: Tableau enables powerful analytics without complex programming
- Interactive Filtering: Real-time brand and range filters provide personalized analysis for each user
- Web Accessible: Flask integration makes dashboards available through any web browser without Tableau Desktop
- Scalable Data Pipeline: The ETL script can be rerun with updated datasets with minimal changes
- Professional-Grade Visuals: Tableau's visualization library produces publication-quality charts
- Free Hosting: Tableau Public provides free cloud hosting for published workbooks
- Multi-Scenario Coverage: The system addresses three distinct user personas with tailored analytical views

17.2 Limitations

- Tableau Public Dependency: Dashboards require internet connectivity to load from Tableau Public servers
- Static Dataset: The current implementation uses a fixed CSV snapshot; real-time EV market updates require manual data refresh
- Tableau Desktop Required: Dashboard creation and editing requires a licensed or trial Tableau Desktop installation
- Limited Predictive Analytics: The current system is descriptive only; no predictive models or forecasting are included
- Browser Compatibility: Some Tableau JS API features may have varying behavior across browsers
- Dataset Scope: The dataset covers European EV market data; global coverage would require expanded data sources

18. Future Enhancements

18.1 Data & Analytics

- Integrate real-time EV market data via automotive APIs (e.g., EV Database API, OEM feeds)
- Add predictive analytics module using Python ML libraries (scikit-learn) to forecast EV range and efficiency trends
- Expand dataset to include global markets (US, Asian EV manufacturers: BYD, NIO, Rivian)
- Include charging infrastructure data (number of charging stations by region) for demand analysis

18.2 Technical Improvements

- Migrate to Tableau Server or Tableau Cloud for enterprise-grade embedding with user authentication
- Implement automated ETL pipeline using Apache Airflow for scheduled data refreshes
- Add user authentication to Flask app with role-based access (planner, manager, consumer views)
- Implement REST API endpoints in Flask for programmatic dashboard data access

18.3 Mobile & Cloud Integration

- Develop mobile-responsive layouts in Tableau using Device Designer for phone/tablet optimization
- Deploy Flask application to cloud platforms (AWS, Heroku, Google Cloud Run) for public access
- Integrate with cloud databases (AWS RDS, Google Cloud SQL) for production-scale data management

18.4 Feature Expansion

- Add CO2 savings calculator: estimate emission reductions by switching from ICE to EV models
- Implement route range analysis: integrate map view to visualize range coverage from a user location
- Add cost-of-ownership calculator: total cost including purchase price, charging costs, and maintenance

19. Conclusion

The Electric Vehicle Market Analysis Dashboard project successfully demonstrates the complete lifecycle of a data analytics application - from raw data collection through structured database storage, interactive Tableau visualization, and web-based delivery via Flask. The project addresses three distinct real-world use cases and provides stakeholders with actionable, visually compelling insights into the electric vehicle market.

From a technical standpoint, the project achieved all defined objectives: a SQL database stores 103 EV records with efficient query support; six unique Tableau visualizations explore key market dimensions; an interactive, filter-enabled dashboard provides multi-perspective analysis; and a Tableau Story presents guided narrative insights. The complete solution is integrated into a Flask web application and deployed to GitHub, demonstrating proficiency in the full data analytics toolchain.

The analytical findings reveal significant market differentiation: Lucid and Lightyear lead in range, Tesla dominates both speed and efficiency metrics, and Segment C vehicles constitute the most accessible entry point into the EV market. The positive correlation between price and range demonstrates the maturity of battery technology investment across market tiers.

Beyond the technical deliverables, this project provided the team with hands-on experience in Tableau as a business intelligence platform, the practical application of data engineering workflows, and the integration of analytics tools into web applications - skills directly applicable to careers in data analytics, business intelligence, and full-stack development.

As electric vehicles continue to reshape global transportation, data-driven platforms like this dashboard will be essential tools for the decision-makers - city planners, fleet operators, consumers, and policy makers - who will shape the EV adoption curve. This project lays a solid, scalable foundation for such a platform.

20. Appendix

20.1 Dataset Details

The project uses the ElectricCarData_Clean.csv dataset containing specifications for electric vehicles available in the European market. The dataset includes the following attributes:

Column Name	Data Type	Description
Brand	String	Vehicle manufacturer name
Model	String	Vehicle model name
AccelSec	Float	0-100 Km/H acceleration time in seconds
TopSpeed_KmH	Integer	Maximum speed in Kilometres per hour
Range_Km	Integer	Full-charge driving range in Kilometres
Efficiency_WhKm	Integer	Energy consumption in Watt-hours per Kilometre
FastCharge_KmH	Integer	Range added per hour via fast charging (Km/H)
RapidCharge	String	Rapid charge support: Yes / No
PowerTrain	String	Drive type: AWD / RWD / FWD
PlugType	String	Charging connector standard (e.g., Type 2 CCS)
BodyStyle	String	Vehicle body type (SUV, Sedan, Hatchback, etc.)
Segment	String	Market segment classification: A, B, C, D, E
Seats	Integer	Number of passenger seats
PriceEuro	Integer	Vehicle price in Euros

20.2 Team & Project Information

Team Member	Role	Responsibilities
Krishna Shankar Maurya	Team Lead	Architecture, Tableau Dashboards, Coordination
Kunal Singh	Member	Flask Integration, Web Development
Prashant Singh	Member	Database Design, Data Pipeline (ETL)
Rudra Tyagi	Member	Testing, Documentation, Story Creation

20.3 Project Stats

- Total Epics: 8
- Total Stories & Tasks: 15

- Total Subtasks: 12
- Total Unique Visualizations: 6
- Dataset Records: 103 EV models
- Brands Covered: 30+
- Dashboard Filters: 2 (Brand multi-select, Range slider)
- Tableau Story Scenes: 4
- Calculated Fields: 5+

20.4 GitHub Repository

The complete source code, Tableau workbook, dataset, and documentation for this project are available on the team's GitHub repository. The repository includes setup instructions in README.md for cloning and running the project locally.

Repository: https://github.com/Krishna-Coder-og/ev-dashboard-project

Electric Vehicle Market Analysis Dashboard

Krishna Shankar Maurya | Kunal Singh | Prashant Singh | Rudra Tyagi

Academic Year 2024-25